Laboratório Funcional · Aula 12

andre mulati · RA 24011149-2 · Turma B
Gerado em 16/10/2026, 19:59:28 · tempo registrado no cronômetro: 27:14

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

1. agora(): é impura porque depende de algo externo, o relógio do sistema. Chamadas diferentes podem devolver datas e horários diferentes, mesmo sem receber parâmetros.

2. registra(msg): é impura porque tem um efeito colateral: escreve uma mensagem na tela usando print(msg). Seu comportamento não consiste apenas em devolver um valor.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2

Saídas (original e com compor(g, f))

```
original
1: A-1 A-2
2: False
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]

modificado
1: A-1 A-2
```

```
2: False
3: 8
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função proximo_id("A") devolve valores diferentes porque depende da variável global contador, que é incrementada a cada chamada. Assim, a primeira chamada retorna "A-1" e a segunda retorna "A-2", mesmo recebendo a mesma entrada.

Isso dificulta os testes porque o resultado depende do estado anterior do programa

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

Porque a ordem em que as funções são aplicadas altera o resultado. Na primeira composição, multiplicamos por 3 e depois somamos 2. Na segunda, somamos 2 primeiro e depois multiplicamos por 3. Portanto, a composição de funções não é necessariamente comutativa:

f
(
g
(
x
)
)
≠
g
(
f
(
x
)
)
f(g(x))
□
=g(f(x)).

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) segue melhor o estilo funcional porque não altera a lista original, evitando efeitos colaterais. Isso torna o código mais previsível e facilita os testes.

Já notas.sort() é uma operação com efeito colateral, pois modifica a lista existente.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
original
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B

sem apostrofo
gera erro por coonta do apostrofo.
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo (`'`) serve para indicar ao Scheme que uma expressão deve ser tratada como dado literal, sem ser executada.

Por exemplo, `(car '(A B C))` funciona porque o apóstrofo faz com que `(A B C)` seja interpretado como uma lista de símbolos. Assim, `car` retorna `A`, o primeiro elemento.

Sem o apóstrofo, em `(car (A B C))`, o Scheme tenta executar `(A B C)` como uma chamada de função, considerando `A` como o nome da função e `B` e `C` como argumentos. Como `A` não está definida como função, ocorre um erro.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))


resultado
C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 3: comum em 6 passos, até 2 multiplicações esperando (expressão de até 24 caracteres); em cauda em 5 passos, nada esperando (até 16 caracteres).
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Sim, vai funcionar, a recursão em cauda permite que implementações Scheme compatíveis executem recursões muito longas com uso constante de pilha. Em Python, para contar até um milhão, é melhor usar um laço while ou for.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

na recursão comum, acumulam-se chamadas pendentes; na recursão em cauda otimizada, esse acúmulo pode ser evitado. Python não faz essa otimização automaticamente, por isso também pode dar erro de profundidade na recursão em cauda.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

o Scheme permite esse tipo de recursão em implementações com otimização de cauda, enquanto o Python tem um limite de profundidade que impede a execução de um milhão de chamadas recursivas.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
; 1) Contar os elementos da lista
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

; 2) Somar os quadrados dos elementos
(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

; 3) Selecionar apenas os números pares
```

```
(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender como a recursão percorre a lista e como cada função trata a lista vazia.

tamanho: usei null? para verificar se a lista está vazia, cdr para avançar para o restante da lista e + 1 para contar cada elemento.

soma-quadrados: usei null? como condição de parada, car para pegar o primeiro número e elevá-lo ao quadrado, e cdr para continuar com os demais números.

pares: usei null? para parar quando a lista acaba, car para verificar se o primeiro número é par com even?, e cdr para percorrer o restante. Usei cons para montar a lista com os números pares encontrados.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

[0,1,4,9,16]

Saídas (original e com member 17)

```
original
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

modificado com member 17
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell consegue imprimir parte de uma lista infinita por causa da avaliação preguiçosa: ele calcula apenas os elementos que são necessários. Assim, take 5 squares retorna [0,1,4,9,16] sem precisar calcular a lista inteira.

A expressão primeiro 42 (div 1 0) não deu erro porque a função primeiro x _ = x ignora o segundo argumento. Como div 1 0 não é necessário para obter o resultado, ele nunca é calculado.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 17 squares nunca termina porque squares contém os quadrados perfeitos em ordem crescente, e 17 não é um quadrado perfeito. A função continua procurando indefinidamente, pois não tem uma condição para desistir da busca.

Já member 16 squares termina porque encontra o número 16 na lista e retorna True.

Como corrigir: podemos parar a busca quando o elemento atual da lista for maior que o número procurado:

```
member b [] = False
member b (a:x)
 | a > b    = False
 | a == b   = True
 | otherwise = member b x
```

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

15

[5, 10, 15]

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Isso acontece por causa de um fechamento (closure). Quando somador(10) é executada, ela devolve outra função que mantém acesso ao parâmetro n, cujo valor é 10, mesmo depois que a execução de somador termina.

Assim, quando chamamos mais10(5), a função usa o 10 guardado e calcula 5 + 10 = 15.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

As funções são aplicadas em ordens diferentes:

f.compose(g).apply(4) → 14: executa primeiro g(4) = 12 e depois f(12) = 14. Corresponde a
f
(
g
(
4
)
)
f(g(4)).

f.andThen(g).apply(4) → 18: executa primeiro f(4) = 6 e depois g(6) = 18. Corresponde a
g
(
f
(
4
)
)
g(f(4)).

O método que corresponde a compor(f, g) da Prática 1 é f.compose(g), pois a composição definida naquela prática calcula primeiro g e depois f.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
1
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();

        System.out.println(soma);
    }
}
saida
244
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
```

```
];

const aprovados = alunos
  .filter((aluno) ⇒ aluno.nota ⩾ 6)
  .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
1 - 244
2 - ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais fácil de ler porque mostra diretamente as operações realizadas nos dados, como filtrar os alunos aprovados e transformar seus nomes em letras maiúsculas. O código fica mais curto e organizado.

Já a versão com laço pode ser mais fácil de depurar em algumas situações, porque permite colocar pontos de interrupção e verificar os valores a cada repetição.

a versão funcional costuma facilitar a leitura, enquanto a versão com laço pode facilitar a depuração passo a passo. Isso depende da complexidade do programa e da familiaridade de quem está lendo o código.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender porque é mais curta e expressa diretamente o que precisa ser feito: gerar a lista de 1 até n, elevar cada número ao cubo e somar os resultados. Já a versão em C exige acompanhar a variável sum, o índice e o laço.

Eu aceitaria o custo de execução em um sistema em que a clareza, a facilidade de manutenção e a confiabilidade fossem mais importantes, como um programa de análise de dados executado ocasionalmente

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi descobrir que o Haskell consegue trabalhar com listas infinitas e evitar cálculos desnecessários graças à avaliação preguiçosa.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O trecho b <- h está errado porque h é o pivô, um único número, e não uma lista da qual b possa ser retirado. O correto é usar a condição b <= h, para selecionar os elementos menores ou iguais ao pivô

O compilador indica que esperava uma lista, mas encontrou um valor numérico. Em outras palavras, inferiu que h deveria ter um tipo de lista, embora o algoritmo use h como pivô numérico. A correção é substituir b <- h por b <= h, que é um teste lógico válido para filtrar os elementos